FACULTY OF ENGINEERING AND TECHNOLOGY

School of Computer Science and Engineering

(Computer Science & Engineering)

Faculty of Engineering & Technology

Jain Global Campus, Kanakapura Taluk - 562112

Ramanagara District, Karnataka, India

AY: 2026-2027

(III Semester)

A Mini Project Report on

DISK SCHEDULER FOR DATABASE WORKLOADS: COMPARISON OF FCFS, SSTF, SCAN AND C-SCAN ON SEEK MOVEMENT AND FAIRNESS

Submitted in partial fulfilment of
Course Code:
Course Name: operating system
for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

NAME-V SREESHARAN
USN-25BTRPW098

Under the guidance of

Dr. N. Satheesh
Professor
Department of Computer Science and Engineering
School of Computer Science & Engineering
Faculty of Engineering & Technology
JAIN (Deemed to-be University)

ABSTRACT

Disk scheduling decides the order in which pending disk requests are served, and it directly affects the speed and fairness of an input/output system. This project presents a disk scheduler simulator for database-like workloads. It generates three types of request patterns (random, sequential and hotspot) and compares four classic algorithms: First Come First Served (FCFS), Shortest Seek Time First (SSTF), SCAN and Circular SCAN (C-SCAN). The algorithms are compared on total seek distance, average wait time and fairness, where fairness is measured using an overtaking metric that counts how many later-arriving requests are served before an earlier one. The simulator was implemented in Python and extended into an interactive web application that animates the disk head. The results show that FCFS gives perfect fairness but the longest head movement, SSTF gives the shortest movement but the worst starvation, and SCAN and C-SCAN give a good balance by keeping head movement low while bounding the waiting time of every request.

INTRODUCTION

A hard disk serves requests by moving its read/write head across tracks. The time spent moving the head, called seek time, is the largest part of disk access time, so the order in which requests are served matters a lot. The operating system uses a disk scheduling algorithm to choose this order.

Database systems produce a special kind of disk traffic. Index lookups tend to cluster around a few hot regions of the disk, table scans read tracks sequentially, and other queries access tracks at random. A scheduler that works well for one pattern may perform badly for another.

This project studies four well-known algorithms: FCFS serves requests in arrival order, SSTF always serves the nearest request, SCAN moves the head in one direction serving requests until the end of the disk and then reverses, and C-SCAN moves in one direction only and jumps back to the start after reaching the end. They are compared on database-like workloads for both efficiency (seek movement) and fairness (starvation).

PROBLEM STATEMENT

To design and implement a simulator that generates random, sequential and hotspot (database-like) disk request workloads, and to compare the FCFS, SSTF, SCAN and C-SCAN disk scheduling algorithms on seek movement and fairness, in order to identify which algorithm is most suitable for database workloads.

PROJECT REQUIREMENTS

Software Requirements
- Operating System: Windows 10/11 (any OS that runs Python and a web browser)
- Programming Language: Python 3.x
- Python Library: matplotlib (for graphs)
- Web Technologies: HTML, CSS and JavaScript (for the interactive simulator)
- Development Environment: Antigravity IDE / VS Code
- Web Browser: Google Chrome or any modern browser

Hardware Requirements
- Standard laptop or desktop with at least 4 GB RAM (no special hardware needed)

METHODOLOGY

1. Block Diagram

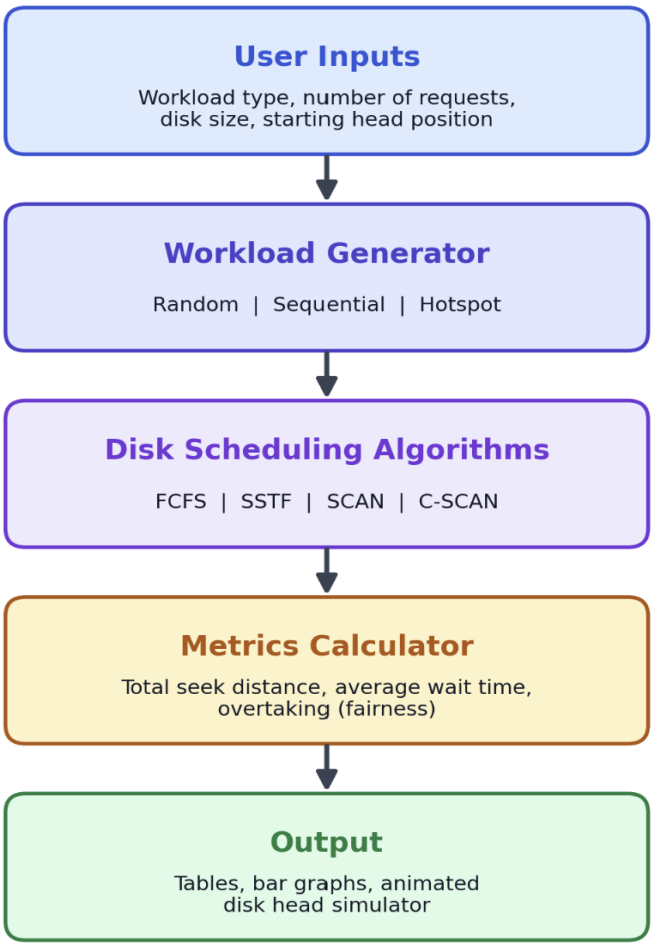


Figure: Block diagram of the Disk Scheduler for Database Workloads

2. Flow of the Project

Step 1: Study the four disk scheduling algorithms and decide the metrics.

Step 2: Build the workload generator for three request types on a disk of 200 tracks:

- Random: tracks chosen uniformly at random.
- Sequential: consecutive tracks starting from a random point.
- Hotspot: 80 percent of requests fall inside 3 small zones of 20 tracks (like database index lookups) and 20 percent are random.

Step 3: Implement FCFS, SSTF, SCAN and C-SCAN. Each algorithm takes the request list and the starting head position and returns the order in which the head travels.

Step 4: Calculate the metrics for each algorithm:

- Total seek distance: sum of the head movement between tracks, including the trip to the disk end in SCAN and C-SCAN and the return jump in C-SCAN.

- Average wait time: average head movement that happened before each request was served.
- Overtaking (fairness): for each request, the number of later-arriving requests that were served before it. The maximum and the average are reported. FCFS always gives 0.

Step 5: Run each algorithm on the same workload, repeat many times and average the results.

Step 6: Display the results as tables and bar graphs (matplotlib).

Step 7: Build the interactive web simulator with an animated disk head, a four-way race mode, a results table with charts and a starvation viewer. Step 8: Analyse the results and draw conclusions.

OUTPUT

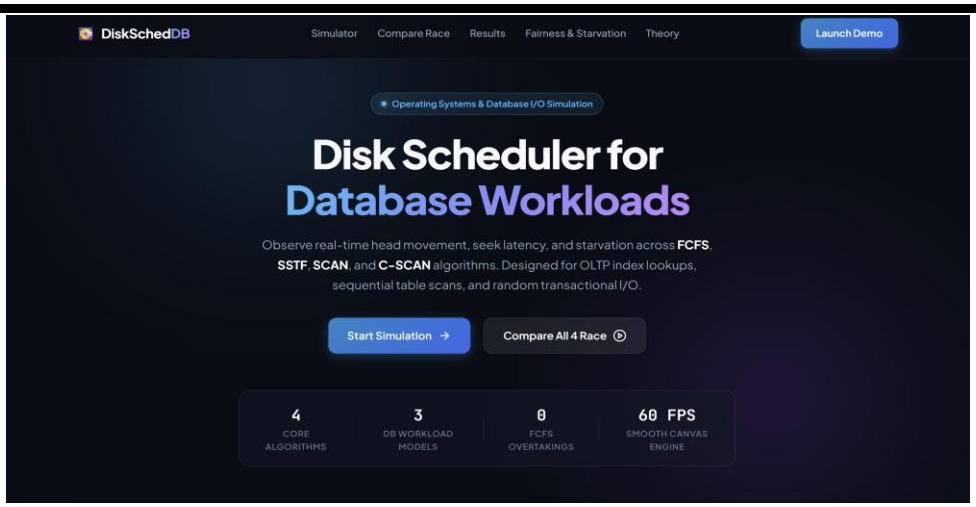
The simulator was run on a hotspot workload with 50 requests, 200 disk tracks and the head starting at track 50. The results are shown below.

Output: Hotspot workload, 50 requests, 200 tracks, head starts at track 50

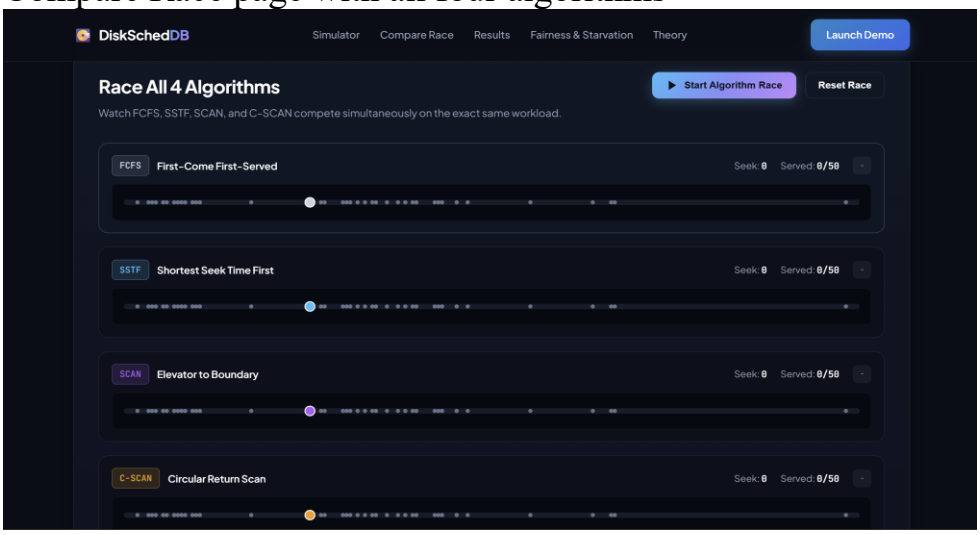
Algorithm	Total Seek Distance	Average Wait Time	Max Overtaking	Avg Overtaking
FCFS	3145.0	1617.1	**0.0**	**0.0**
SSTF	**232.0**	**89.1**	47.0	9.9
SCAN	342.0	201.8	38.0	12.7
C-SCAN	392.0	230.4	44.0	14.4

Green = best value in column (lower is better)

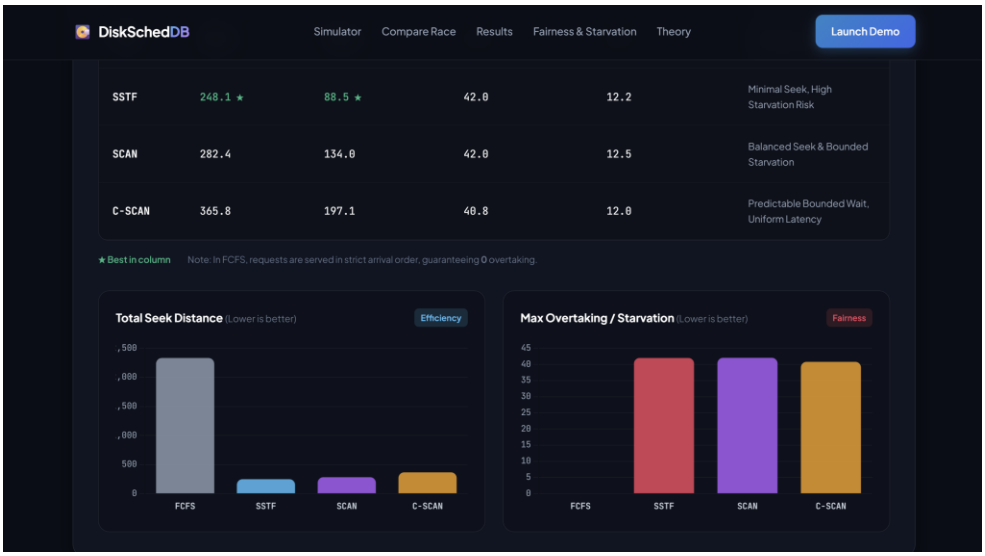
Simulator page showing controls and live counters



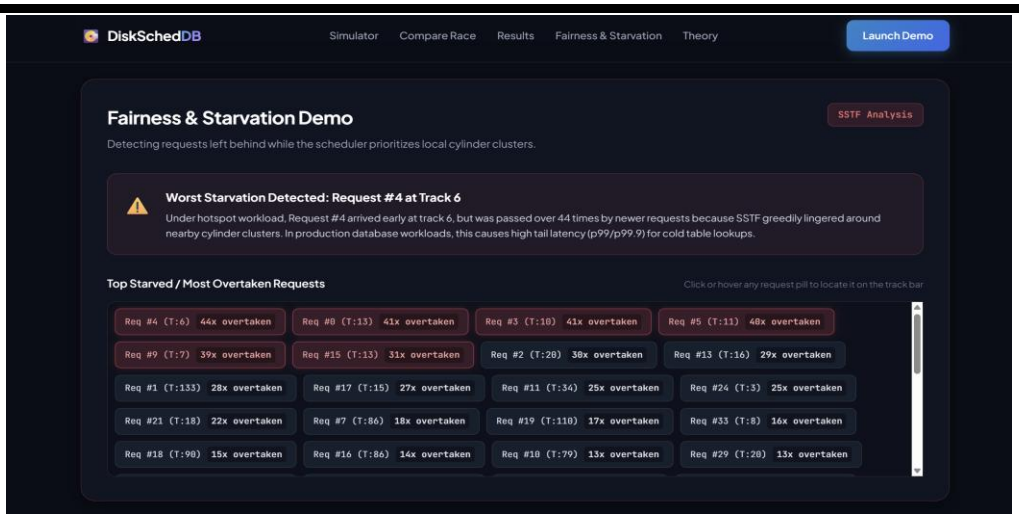
Compare Race page with all four algorithms



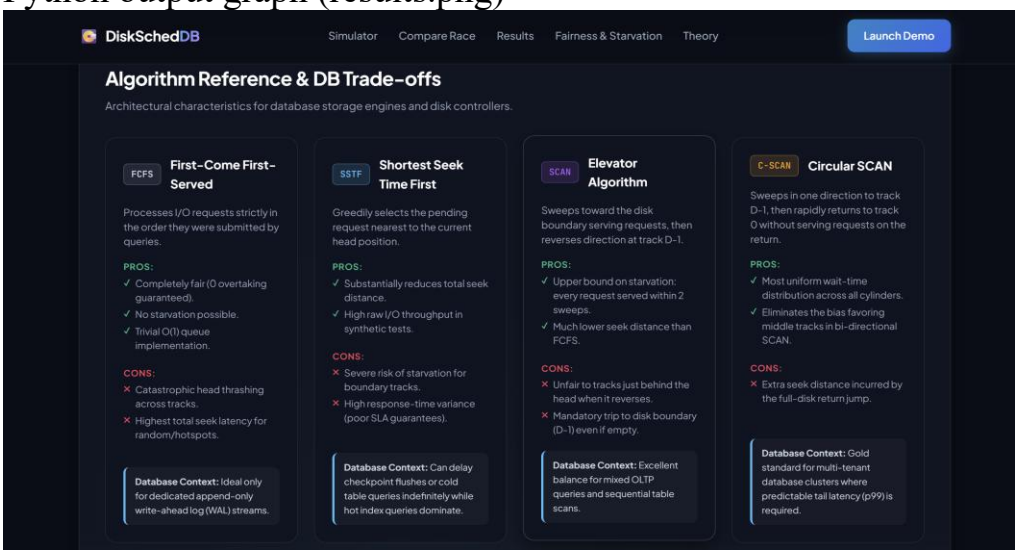
Results table and bar charts



Fairness and Starvation page



Python output graph (results.png)



Observations:

1. FCFS moved the head the most (3145 tracks) because it serves requests in arrival order and jumps across the disk. However it has 0 overtaking, so it is perfectly fair.
2. SSTF moved the head the least (232 tracks) and has the lowest average wait, but it has the worst starvation. In this run, Request 0 at track 170 was overtaken 47 times because SSTF kept serving nearer requests in the hotspot zones.
3. SCAN (342) and C-SCAN (392) moved far less than FCFS. They also guarantee that every request is served within one or two sweeps of the disk, so waiting time is bounded.
4. C-SCAN has a slightly higher seek distance than SCAN because of the return jump to track 0, but it gives a more uniform wait across all tracks

5. On sequential workloads, all algorithms perform well because the requests are already in order. Note: The values change for every randomly generated workload, so the final comparison was based on averaging multiple runs.

CONCLUSION

This project compared FCFS, SSTF, SCAN and C-SCAN on random, sequential and hotspot database-like workloads using total seek distance, average wait time and an overtaking-based fairness metric. FCFS is completely fair but very slow on random and hotspot workloads. SSTF gives the shortest head movement but can starve requests that are far from the current head position, which would cause high tail latency for database queries. SCAN and C-SCAN keep head movement close to SSTF while giving every request a bounded waiting time. Therefore SCAN or C-SCAN is the best choice for database workloads where both throughput and predictable response time matter, with C-SCAN preferred when uniform waiting across all tracks is required. The simulator and the animated web application make these differences easy to see and can be extended with algorithms such as LOOK and C-LOOK.